

Code-Layout, Readability and Reusability

Date	28 June 2026
Team ID	XXXXXX
Project Name	Rising Waters
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used through Out the code	Yes	Proper indentation maintained throughout the project.
2	Proper File Structure	Files and folders are logically organized	Yes	Backend, frontend, configuration files, and deployment files are logically organized.
3	Meaningful Variable Name	Variables reflect their purpose clearly	Yes	Variables such as annual rainfall, humidity, cloud visibility clearly indicate their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions like generate_demo_model() and prediction-related methods use descriptive names.
5	Code Comments	Inline and block comments explain logic	Yes	Important sections include comments for better understanding and maintenance.
6	Modular Design	Code is split into reusable functions/modules	Yes	Authentication, prediction engine, routes, and templates are implemented as separate modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Duplicate logic has been minimized by reusing common functions and modules.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Input validation and exception handling improve application reliability.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Authentication Module	Flask	User Login, Signup, Session Management	High
2	Data Preprocessing	Python, Pandas	Data Cleaning, Feature Preparation, Model Training	High
3	Prediction Engine	XGBoost	Flood Risk Prediction, Model Inference	High
4	Database Module	SQLite	User Data, Prediction History	High
5	Web Interface	HTML, CSS, JavaScript	Dashboard, Prediction Form, Result Pages	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Well-organized project structure with separate backend, frontend, and configuration files.
Readability	5	Meaningful variable names, proper indentation, and consistent coding style.
Reusability	5	Modular components such as authentication, preprocessing, prediction engine, and database can be reused.
Documentation / Comments	5	README and code comments make the project easy to understand and maintain.
Overall Score	5	Clean, modular, maintainable, and scalable codebase suitable for future enhancements.